MongoDB: documentos cuando la fila no es la forma natural

Equipo 3 · Heidi, Scarlet, Damian

1. Tecnología elegida y problema que resuelve

Tecnología: MongoDB, una base de datos orientada a documentos. Cada registro es un documento JSON (guardado en formato BSON) y los documentos se agrupan en colecciones.

En el Sistema General de Ingresos hay datos que no tienen forma de fila. Una licencia de funcionamiento trae giro y metros cuadrados; una constancia de no adeudo trae RFC y número de adeudos. Si mañana aparece un trámite nuevo con tres campos distintos, en MySQL hay que hacer `ALTER TABLE` y dejar columnas en `NULL` para los demás tipos.

MongoDB guarda cada trámite como **un documento completo**, con los campos que ese trámite realmente tiene, dentro de la misma colección.

Problema en una frase: guardar datos con forma variable sin modificar la tabla cada vez que aparece un tipo nuevo.

2. Cómo se usa, en pasos

1. **Elegir base y colección.** Por ejemplo, base `ingresos` y colección `tramites`.
2. **Armar el documento.** Un objeto JSON con los campos de ese trámite: `tipo`, `folio`, `fecha`, `estado` y un objeto `datos` que cambia según el tipo.
3. **Insertarlo.** `db.tramites.insertOne(doc)`. No hay que declarar columnas antes.
4. **Consultarlo.** `db.tramites.find({ "datos.giro": "restaurante" })`. Se puede buscar dentro del objeto anidado.
5. **Indexar los campos que se consultan.** `db.tramites.createIndex({ "datos.rfc": 1 })` para no recorrer toda la colección en cada búsqueda.

En la API, el cliente (por ejemplo `pymongo` en Python o `mongoose` en Node) hace las mismas operaciones: `insert_one`, `find_one`, `update_one`.

3. Piezas con las que se trabaja

Pieza	Qué es	En el proyecto
Servidor	Proceso `mongod` que guarda los documentos	Contenedor `mongo:7`, puerto 27017
Cliente	Librería que usa la API	`pymongo` (Python) o `mongoose` (Node)
Shell	`mongosh`, consola interactiva	`use`, `insertOne`, `find`, `updateOne`
Colección	Grupo de documentos	`tramites`, `pagos`, `bitacora`
Índice	Estructura para acelerar consultas	`createIndex({ "datos.rfc": 1 })`
Atlas	MongoDB en la nube	Alternativa sin instalar servidor

Aclaración importante: MongoDB no es la fuente de verdad de todo el sistema. Los pagos, la contabilidad y las relaciones entre entidades siguen en MySQL. MongoDB guarda lo que no tiene forma de tabla.

4. Parecidos y diferencias con lo que ya vimos en el curso

Ya lo vimos	Cómo funciona	Qué hace MongoDB
MySQL	Filas y columnas con esquema fijo	Documentos con campos variables
Docker	Levanta la API y la base de datos	Es un contenedor más: `docker run mongo:7`
GraphQL	Define qué datos pide el cliente	No es una API: es almacenamiento interno
Token / permisos	Deciden quién puede entrar	No autentica: la API sigue validando

La diferencia clave es el modelo. MySQL guarda **filas** y las relaciona con `JOIN`; MongoDB guarda **documentos** y espera que lo que se lee junto esté junto. A cambio, pierde los `JOIN` y las relaciones fuertes, que en MySQL son gratuitas.

5. Ejemplo concreto

Dos documentos en la **misma** colección, con campos distintos:

```
// licencia de funcionamiento
{
  "tipo": "licencia",
  "folio": "L-1042",
  "fecha": "2026-10-20",
  "datos": { "giro": "restaurante", "m2": 85 }
}

// constancia de no adeudo
{
  "tipo": "constancia",
  "folio": "C-2087",
  "fecha": "2026-10-21",
  "datos": { "rfc": "XAXX010101000", "adeudos": 0 }
}
```

Consulta al campo anidado:

```
db.tramites.find({ "datos.giro": "restaurante" })
```

En MySQL esto serían dos tablas distintas, o una sola tabla con la mitad de las columnas en `NULL`. En MongoDB no hubo que modificar nada para aceptar el segundo tipo.

6. Qué gana el equipo y qué costo tiene

Gana:

- Un trámite se lee completo en una sola consulta, sin `JOIN`.
- Esquema flexible: cada tipo trae sus propios campos.
- Menos migraciones: no hay `ALTER TABLE` por cada tipo nuevo.
- Escala horizontal con réplicas para lecturas.

Cuesta:

- Duplicar datos: lo que se repite hay que mantenerlo en varios documentos.
- Sin `JOIN`: las relaciones se resuelven en la aplicación.
- Transacciones más limitadas que en MySQL al tocar varias colecciones.
- Otro modelo que aprender; el equipo ya piensa en SQL.
- El esquema no desaparece: se diseña en la aplicación, no en la base.

7. Cuándo no conviene usarla

- **Datos muy relacionados** (pagos, contabilidad, nómina): el `JOIN` es la herramienta correcta y en MongoDB habría que duplicar.
- **Reportes que cruzan muchas entidades**: sumar por sucursal, mes y tipo es más simple en SQL.
- **Transacciones que tocan varias entidades a la vez**: si un movimiento afecta cinco tablas, conviene una base relacional.
- **Esquema fijo y simple**: si los datos ya caben en una tabla, MySQL funciona y el equipo ya lo sabe usar.

La pregunta guía es: **¿este dato se lee completo y cambia por bloques?**

8. Demo (opcional, máximo 3 minutos)

Tiempo	Acción	Comando
0:00	Levantar MongoDB	`docker run -d --name mongo -p 27017:27017 mongo:7`
0:40	Abrir consola	`mongosh` → `use ingresos`
1:10	Insertar trámite tipo licencia	`db.tramites.insertOne({...})`
1:40	Insertar trámite tipo constancia	`db.tramites.insertOne({...})` con otros campos
2:10	Consultar campo anidado	`db.tramites.find({ "datos.giro": "restaurante" })`
2:40	Cierre	Misma colección, sin `ALTER TABLE`

Plan B si falla la red: video grabado de 3 minutos. El PDF de la actividad lo permite.

9. Fuentes

1. MongoDB. *MongoDB Manual*. https://www.mongodb.com/docs/manual/
2. MongoDB. *Data Modeling Introduction*. https://www.mongodb.com/docs/manual/core/data-modeling-introduction/
3. Docker Hub. *mongo (official image)*. https://hub.docker.com/_/mongo

10. Reparto de la exposición (15 minutos)

Tiempo	Quién	Qué dice
0:00–0:40	Scarlet	Saludo, equipo y orden de la exposición
0:40–3:30	Heidi	El problema: el trámite que no cabe en una tabla
3:30–5:30	Heidi	Qué es MongoDB y sus piezas
5:30–7:30	Scarlet	Cómo se usa: base, colección, documento e índice
7:30–9:30	Damian	Ejemplo concreto: dos trámites, una colección
9:30–11:00	Damian	Comparación con MySQL, Docker, GraphQL y el token
11:00–13:30	Scarlet	Costos, límites y cuándo no conviene
13:30–14:30	Damian	Demo
14:30–15:00	Heidi	Cierre y preguntas

> *Ajustar los nombres si el reparto real es distinto. Cada integrante debe poder explicar cualquier diapositiva: el jurado puede preguntar a cualquiera.*